Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

Week 5

Advanced Spark Programming

Objective 1 - Conceptual Foundations

Spark SQL allows users to write familiar SQL queries while executing them on top of Spark's distributed processing engine. Rather than running on a single machine, Spark SQL leverages Spark's architecture to process large datasets efficiently across a cluster.

When a query is submitted, Spark first parses and analyzes it to understand its structure and intent. The query is then passed to the Catalyst Optimizer, which inspects the logical plan and applies optimizations such as removing redundant operations, simplifying expressions, and reordering joins to improve performance.

After optimization, Spark determines the most efficient physical execution plan. This plan defines how the work will be distributed across multiple machines, allowing the query to run in parallel and scale to large volumes of data while maintaining fault tolerance.

Spark SQL can read from a wide range of data sources, including Parquet files, CSVs, and relational databases. It is a foundational technology behind many modern big-data platforms such as data lakes and Databricks, helping bridge the gap between massive datasets and end users by making large-scale data accessible through SQL.

Objective 2 – Spark SQL with Scala

1. This loads a CSV file into Spark and makes it available as a SQL table so it can be queried like a database.
   The successful queries and results show that Spark can read data from HDFS and run SQL queries against it correctly.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

```
scala>   val df = spark.read.format("csv").option("header", "true").load("/data/grades.csv")
df: org.apache.spark.sql.DataFrame = [Last name: string, First name: string ... 7 more fields]

scala> df.createOrReplaceTempView("df")

scala> spark.sql("SHOW TABLES").show()
54924 [main] WARN  org.apache.hadoop.hive.conf.HiveConf  - HiveConf of name hive.strict.managed.tabl
es does not exist
54924 [main] WARN  org.apache.hadoop.hive.conf.HiveConf  - HiveConf of name hive.create.as.insert.on
ly does not exist
54947 [main] WARN  org.apache.spark.sql.hive.client.HiveClientImpl  - Detected HiveConf hive.executi
on.engine is 'tez' and will be reset to 'mr' to disable useless hive logic
+--------+-------------------+-----------+
|database|          tableName|isTemporary|
+--------+-------------------+-----------+
| default|             grades|      false|
| default|soil_nutrient_pools|      false|
|        |                 df|       true|
+--------+-------------------+-----------+


scala> spark.sql("SELECT * FROM df WHERE Final > 50").show()
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|Last name|First name|        SSN|Test1|Test2|Test3|Test4|Final|Grade|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|  Airpump|    Andrew|223-45-6789|   49|    1|   90|  100|   83|    A|
|   Backus|       Jim|143-12-1234|   48|    1|   97|   96|   97|   A+|
| Elephant|       Ima|456-71-9012|   45|    1|   78|   88|   77|   B-|
| Franklin|     Benny|234-56-2890|   50|    1|   90|   80|   90|   B-|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+


scala> spark.sql("SELECT * FROM df").show()
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|Last name|First name|        SSN|Test1|Test2|Test3|Test4|Final|Grade|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|  Alfalfa|  Aloysius|123-45-6789|   40|   90|  100|   83|   49|   D-|
|   Alfred|University|123-12-1234|   41|   97|   96|   97|   48|   D+|
|    Gerty|    Gramma|567-89-0123|   41|   80|   60|   40|   44|    C|
|  Android|  Electric|087-65-4321|   42|   23|   36|   45|   47|   B-|
|  Bumpkin|       Fred|456-78-9012|   43|   78|   88|   77|   45|   A-|
|   Rubble|     Betty|234-56-7890|   44|   90|   80|   90|   46|   C-|
|   Noshow|     Cecil|345-67-8901|   45|   11|   -1|    4|   43|    F|
|     Buff|       Bif|632-79-9939|   46|   20|   30|   40|   50|   B+|
|  Airpump|    Andrew|223-45-6789|   49|    1|   90|  100|   83|    A|
|   Backus|       Jim|143-12-1234|   48|    1|   97|   96|   97|   A+|
|Carnivore|       Art|565-89-0123|   44|    1|   80|   60|   40|   D+|
|    Dandy|       Jim|087-75-4321|   47|    1|   23|   36|   45|   C+|
| Elephant|       Ima|456-71-9012|   45|    1|   78|   88|   77|   B-|
| Franklin|     Benny|234-56-2890|   50|    1|   90|   80|   90|   B-|
|   George|       Boy|345-67-3901|   40|    1|   11|   -1|    4|    B|
|Heffalump|    Harvey|632-79-9439|   30|    1|   20|   30|   40|    C|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+


scala>
```

2. Run 3 additional SQL commands
    a. spark.sql("SELECT Grade, COUNT(*) FROM df GROUP BY Grade").show(). This command counts the grades per group, showcasing Spark SQL's ability to group across the cluster.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

```
scala> spark.sql("SELECT Grade, COUNT(*) FROM df GROUP BY Grade").show()
+-----+--------+
|Grade|count(1)|
+-----+--------+
|    F|       1|
|    B|       1|
|   A-|       1|
|   D+|       2|
|   C+|       1|
|    C|       2|
|   C-|       1|
|   D-|       1|
|    A|       1|
|   B+|       1|
|   A+|       1|
|   B-|       3|
+-----+--------+


scala>

scala>
```

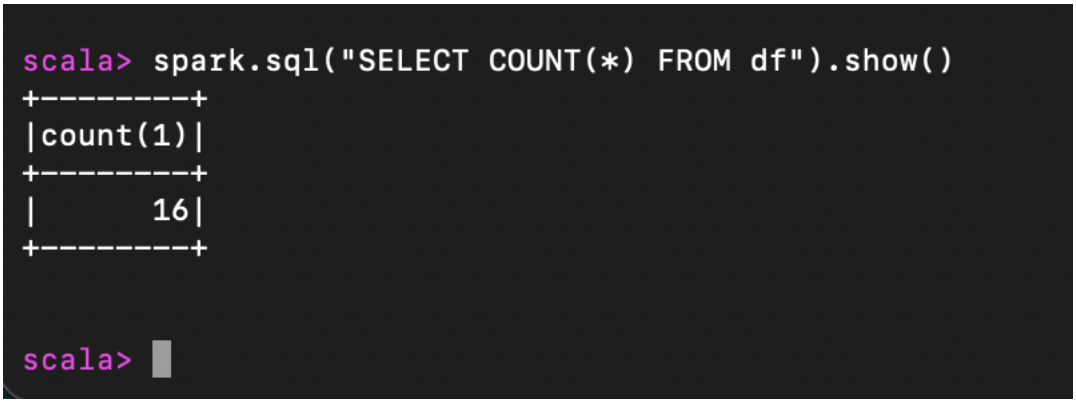
b. spark.sql("SELECT `Last name`, `First name`, Final FROM df ORDER BY Final DESC").show(). This command selects columns to display cleaner results but also orders by score.

```
scala> spark.sql("SELECT `Last name`, `First name`, Final FROM df ORDER BY Final DESC").show();
+---------+----------+-----+
|Last name|First name|Final|
+---------+----------+-----+
|   Backus|       Jim|   97|
| Franklin|     Benny|   90|
|  Airpump|    Andrew|   83|
| Elephant|       Ima|   77|
|     Buff|       Bif|   50|
|  Alfalfa|  Aloysius|   49|
|   Alfred|University|   48|
|  Android|  Electric|   47|
|   Rubble|     Betty|   46|
|    Dandy|       Jim|   45|
|  Bumpkin|      Fred|   45|
|    Gerty|    Gramma|   44|
|   Noshow|     Cecil|   43|
|Carnivore|       Art|   40|
|Heffalump|    Harvey|   40|
|   George|       Boy|    4|
+---------+----------+-----+


scala>
```

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

c. spark.sql("SELECT COUNT(*) FROM df").show(). This last command simply counts the number of rows within the data frame.



Objective 3 – Spark SQL with Python

We are now going to run the same commands in Python. This demonstrates the similar nature of scala and Python. We must remember that a major benefit of Spark SQL is its usability across different languages and technologies, enabling people from different backgrounds to pick up Spark SQL with ease.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

```
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 3.0.0
      /_/

Using Python version 3.7.10 (default, Mar  2 2021 09:06:08)
SparkSession available as 'spark'.
>>>     df = spark.read.format('csv').option('header', 'true').load('/data/grades.csv')
  File "<stdin>", line 1
    df = spark.read.format('csv').option('header', 'true').load('/data/grades.csv')
    ^
IndentationError: unexpected indent
>>> df = spark.read.format('csv').option('header', 'true').load('/data/grades.csv')
>>> df.show()
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|Last name|First name|        SSN|Test1|Test2|Test3|Test4|Final|Grade|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|  Alfalfa|  Aloysius|123-45-6789|   40|   90|  100|   83|   49|   D-|
|   Alfred|University|123-12-1234|   41|   97|   96|   97|   48|   D+|
|    Gerty|    Gramma|567-89-0123|   41|   80|   60|   40|   44|    C|
|  Android|  Electric|087-65-4321|   42|   23|   36|   45|   47|   B-|
|  Bumpkin|      Fred|456-78-9012|   43|   78|   88|   77|   45|   A-|
|   Rubble|     Betty|234-56-7890|   44|   90|   80|   90|   46|   C-|
|   Noshow|     Cecil|345-67-8901|   45|   11|   -1|    4|   43|    F|
|     Buff|       Bif|632-79-9939|   46|   20|   30|   40|   50|   B+|
|  Airpump|    Andrew|223-45-6789|   49|    1|   90|  100|   83|    A|
|   Backus|       Jim|143-12-1234|   48|    1|   97|   96|   97|   A+|
|Carnivore|       Art|565-89-0123|   44|    1|   80|   60|   40|   D+|
|    Dandy|       Jim|087-75-4321|   47|    1|   23|   36|   45|   C+|
| Elephant|       Ima|456-71-9012|   45|    1|   78|   88|   77|   B-|
| Franklin|     Benny|234-56-2890|   50|    1|   90|   80|   90|   B-|
|   George|       Boy|345-67-3901|   40|    1|   11|   -1|    4|    B|
|Heffalump|    Harvey|632-79-9439|   30|    1|   20|   30|   40|    C|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+

>>> df.createOrReplaceTempView('df')
>>> spark.sql('SHOW TABLES').show()
51265 [Thread-4] WARN  org.apache.hadoop.hive.conf.HiveConf  - HiveConf of name hive.strict.managed.tab
les does not exist
51265 [Thread-4] WARN  org.apache.hadoop.hive.conf.HiveConf  - HiveConf of name hive.create.as.insert.o
nly does not exist
51282 [Thread-4] WARN  org.apache.spark.sql.hive.client.HiveClientImpl  - Detected HiveConf hive.execut
ion.engine is 'tez' and will be reset to 'mr' to disable useless hive logic
+--------+------------------+-----------+
|database|         tableName|isTemporary|
+--------+------------------+-----------+
| default|            grades|      false|
| default|soil_nutrient_pools|      false|
|        |                df|       true|
+--------+------------------+-----------+
```

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

```
>>> spark.sql('SELECT * FROM df WHERE Final > 50').show()
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|Last name|First name|        SSN|Test1|Test2|Test3|Test4|Final|Grade|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|  Airpump|    Andrew|223-45-6789|   49|    1|   90|  100|   83|    A|
|   Backus|       Jim|143-12-1234|   48|    1|   97|   96|   97|   A+|
| Elephant|       Ima|456-71-9012|   45|    1|   78|   88|   77|   B-|
| Franklin|     Benny|234-56-2890|   50|    1|   90|   80|   90|   B-|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+

>>> spark.sql('SELECT * FROM df').show()
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|Last name|First name|        SSN|Test1|Test2|Test3|Test4|Final|Grade|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
|  Alfalfa|  Aloysius|123-45-6789|   40|   90|  100|   83|   49|   D-|
|   Alfred|University|123-12-1234|   41|   97|   96|   97|   48|   D+|
|    Gerty|    Gramma|567-89-0123|   41|   80|   60|   40|   44|    C|
|  Android|  Electric|087-65-4321|   42|   23|   36|   45|   47|   B-|
|  Bumpkin|       Fred|456-78-9012|   43|   78|   88|   77|   45|   A-|
|   Rubble|     Betty|234-56-7890|   44|   90|   80|   90|   46|   C-|
|   Noshow|     Cecil|345-67-8901|   45|   11|   -1|    4|   43|    F|
|     Buff|       Bif|632-79-9939|   46|   20|   30|   40|   50|   B+|
|  Airpump|    Andrew|223-45-6789|   49|    1|   90|  100|   83|    A|
|   Backus|       Jim|143-12-1234|   48|    1|   97|   96|   97|   A+|
|Carnivore|       Art|565-89-0123|   44|    1|   80|   60|   40|   D+|
|    Dandy|       Jim|087-75-4321|   47|    1|   23|   36|   45|   C+|
| Elephant|       Ima|456-71-9012|   45|    1|   78|   88|   77|   B-|
| Franklin|     Benny|234-56-2890|   50|    1|   90|   80|   90|   B-|
|   George|       Boy|345-67-3901|   40|    1|   11|   -1|    4|    B|
|Heffalump|    Harvey|632-79-9439|   30|    1|   20|   30|   40|    C|
+---------+----------+-----------+-----+-----+-----+-----+-----+-----+
```

Three commands run in Spark SQL :

1. spark.sql("SELECT COUNT(*) FROM df").show();
    a. screenshot

```
>>> spark.sql("SELECT COUNT(*) FROM df").show();
+--------+
|count(1)|
+--------+
|      16|
+--------+
```

    b. Explanation – This is the same command we ran in Scala. We are counting the number of rows in the data frame.

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

2. `spark.sql("SELECT `Last name`, `First name`, Final FROM df ORDER BY Final DESC").show()`

    a. Screenshot

```
>>> spark.sql("SELECT `Last name`, `First name`, Final FROM df ORDER BY Final DESC").show()
+---------+----------+-----+
|Last name|First name|Final|
+---------+----------+-----+
|   Backus|       Jim|   97|
| Franklin|     Benny|   90|
|  Airpump|    Andrew|   83|
| Elephant|       Ima|   77|
|     Buff|       Bif|   50|
|  Alfalfa|  Aloysius|   49|
|   Alfred|University|   48|
|  Android|  Electric|   47|
|   Rubble|     Betty|   46|
|    Dandy|       Jim|   45|
|  Bumpkin|      Fred|   45|
|    Gerty|    Gramma|   44|
|   Noshow|     Cecil|   43|
|Carnivore|       Art|   40|
|Heffalump|    Harvey|   40|
|   George|       Boy|    4|
+---------+----------+-----+
```

    b. Like the scala command, this query selects only 3 columns (first and last name with the final grade) before ordering by final grade, descending

3. spark.sql("SELECT Grade, COUNT(*) FROM df GROUP BY Grade").show()

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

a. Screenshot

```
>>> spark.sql("SELECT Grade, COUNT(*) FROM df GROUP BY Grade").show()
+-----+--------+
|Grade|count(1)|
+-----+--------+
|    F|       1|
|    B|       1|
|   A-|       1|
|   D+|       2|
|   C+|       1|
|    C|       2|
|   C-|       1|
|   D-|       1|
|    A|       1|
|   B+|       1|
|   A+|       1|
|   B-|       3|
+-----+--------+

>>>
```

b. Explanation – Selects the grades as groups. This showcases the ability to aggregate groups and count their existence within the data set.

Objective 4 – Spark SQL with Custom Data

For the final objective, we will use custom data from the soil dataset from Kaggle. Because it's still within the machine from last week, we'll simply create a table and create the view.

Our first query is to show the number of rows, which is 1006

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

```
pyspark.sql.utils.AnalysisException: Path does not exist: hdfs://master:9000/user/root/data/soil/soil_nutrie
nt_pools.csv;
>>> soil_df = spark.read.format('csv').option('header', 'true').load('/data/soil/soil_nutrient_pools.csv')
>>> soil_df.columns
['SampleID', 'site', 'block', 'paddock', 'plot', 'slope', 'rainfall_reduction', 'grazing_treatment', 'year',
 'type', 'faith_pd']
>>> soil_df.createOrReplaceTempView("soil")
>>> spark.sql("""
... SELECT COUNT(*) AS total_samples
... FROM soil
... """).show()
+-------------+
|total_samples|
+-------------+
|         1006|
+-------------+

>>>
```

Next, we'll print the number of samples per site.

```
>>> spark.sql("""
... SELECT site, COUNT(*) AS sample_count
... FROM soil
... GROUP BY site
... ORDER BY sample_count DESC
... """).show()
+----+------------+
|site|sample_count|
+----+------------+
|  FK|         520|
|  TB|         486|
+----+------------+

>>>
```

Gabriel Vance
DSC650 – Big Data
Professor Nasheb Ismaily
January 25, 2026

Finally our last query groups the soil data by year, counts how many samples were collected each year, and computes the average Faith PD for each year.
Because the Faith PD values are stored as strings with dot separators, the query first removes the dots and converts the values to numbers, which allows Spark to calculate a valid yearly average biodiversity value.

```
>>> spark.sql("""
... SELECT year,
...        COUNT(*) AS samples,
...        AVG(CAST(regexp_replace(faith_pd, '\\\\.', '') AS DOUBLE)) AS avg_faith_pd
... FROM soil
... WHERE faith_pd IS NOT NULL
... GROUP BY year
... ORDER BY year
... """).show()
+----+-------+--------------------+
|year|samples|        avg_faith_pd|
+----+-------+--------------------+
|2018|    203| 3.774465559408867E9|
|2019|    212|3.7031836041792455E9|
|2020|    215| 3.158955000544186E9|
|2021|    204| 3.183662239882353E9|
|2022|    172|3.1264042770174417E9|
+----+-------+--------------------+

>>>
```